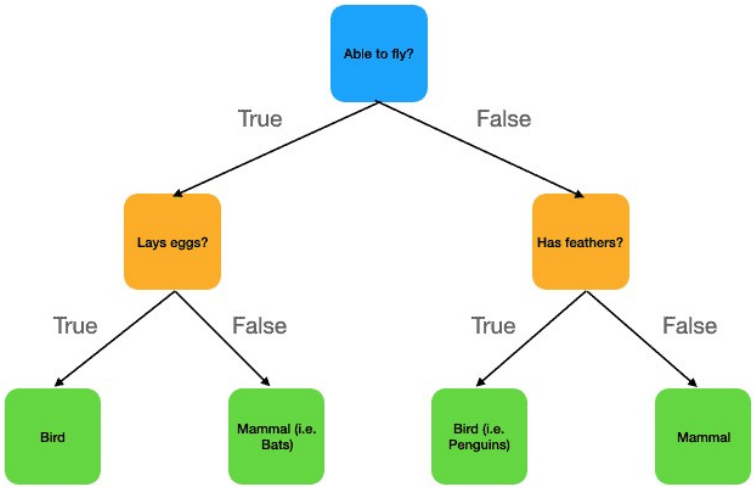
ESL 5

The Elements of Statistical Learning

Decision Tree & Random Forest

Ján Sliacky
Laboratories Brainworks
Faculty of Informatics and Information Technologies STU

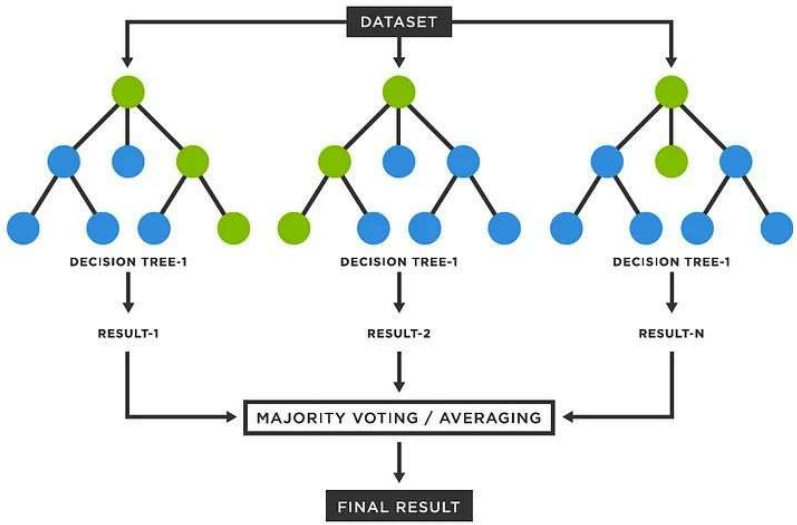


Decision Tree

A decision tree is a **decision support recursive partitioning structure** that uses a tree-like model of decisions and their possible consequences, including chance event outcomes, resource costs, and utility. It is one way to display an algorithm that **only contains conditional control statements.**

Random Forest

Random forests or random decision forests is an **ensemble learning method for classification, regression and other tasks** that works by creating a **multitude of decision trees** during training. For classification tasks, the output of the random forest is the class selected by most trees. For regression tasks, the output is the average of the predictions of the trees. Random forests **correct** for decision trees' habit of **overfitting** to their training set.



	Decision Trees	Random Forest
Response Variable	Binary or multiclass	Binary or multiclass

Predictor Variable(s)

	Decision Trees	Random Forest
Independence	Observations should be independent	Observations should be independent
Collinearity	No restriction (can handle multicollinearity)	No restriction, but may affect feature importance
Normality	Not required	Not required
Feature Scaling	Not required	Not required

Other Assumptions

	Decision Trees	Random Forest
Homoscedasticity	Not required	Not required
Outliers	Robust to outliers	Robust to outliers
Sample Size	Small datasets may overfit; pruning recommended	Performs better with larger datasets
Linear Relationship	Not required (captures non-linear patterns)	Not required (captures non-linear patterns)

Classification settings

What about class imbalance?
In theory, the classes do not have to be similar in size.

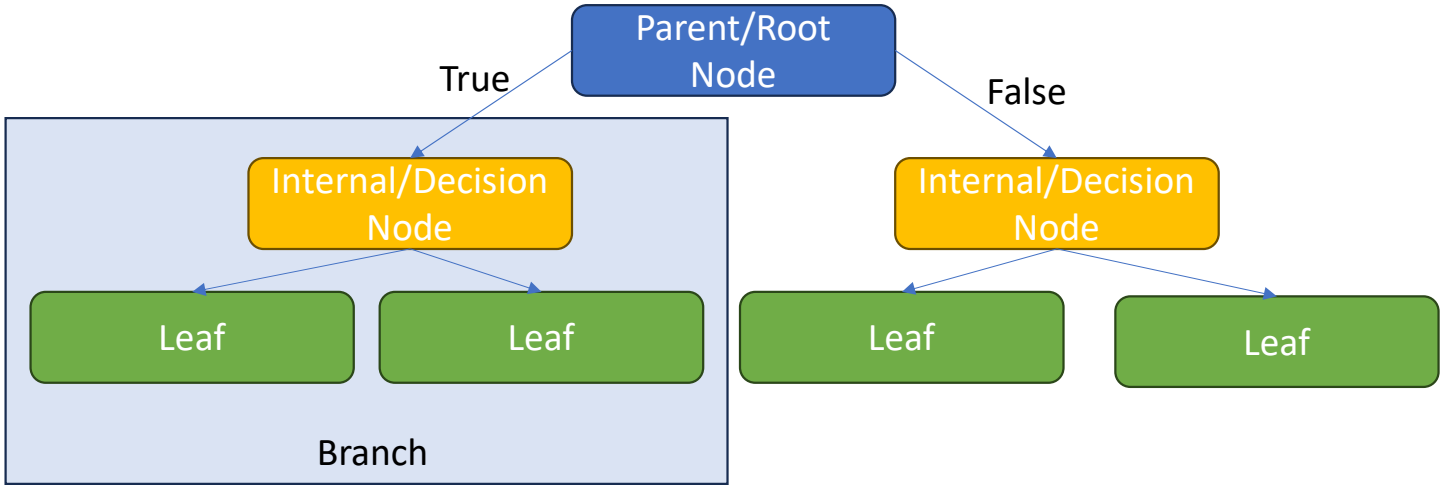
However, a class imbalance can still affect performance. Model ma favor the majority class and minority class may be poorly predicted (remember NIR).

Random Forest is slightly better than single tree but still can bias toward majority class.

How to handle imbalance?
- Class weights
- Balanced random forest
- Resampling
  - Oversampling (SMOTE)
  - Undersampling
- Use better metrics:
  - F1 score
  - ROC – AUC
  - Precision / Recall

Classes do not need to be balanced for Decision Trees or Random Forests, but **strong class imbalance can bias the model toward the majority class and reduce performance on minority classes.**

The anatomy of a decision tree



Building a Decision Tree

1. Select the Best Feature for Splitting
2. Split the Dataset
3. Recursively Repeat Steps 1. & 2.
4. Tree Pruning (Stopping Conditions)

How to select the Best Feature for Splitting

The decision tree starts at the root and selects the best feature to split the dataset. The selection is based on a **splitting criterion**, such as:

Classification

Gini Impurity

$$Gini = 1 - \sum p_i^2$$

Entropy (Information Gain)

$$H(S) = -\sum p_i \log_2 p_i$$

Regression

MSE (Mean Squared Error)

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Gini Impurity and Entropy: intuition

How to understand?

Gini Impurity

$$Gini = 1 - \sum p_i^2$$

If k = 2, Gini interval is [0, 0.5]
If Yes probability = 1, then Gini = 1 – 1 – 0 = 0
If Yes probability = 0.5, then Gini = 1 – 0.25 – 0.25 = 0.5

General interval [0, 1-1/k], when k = number of classes

Lower the Gini impurity the better.

Entropy (Information Gain)

$$H(S) = -\sum p_i \log_2 p_i$$

If k = 2, Entropy interval is [0,1]
If Yes probability = 1, then H = -(1.$\log_2$1 + 0.$\log_2$0) = 0
If Yes probability =0.5, then H = -(0.5.$\log_2$0.5 + 0.5.$\log_2$0.5) = -(0.5*(-1) + 0.5*(-1)) = 1

General interval [0, $\log_2$k]
Maximum entropy occurs when all classes are equally likely.

Lower Entropy the better (more pure node)

How to select the Best Feature for Splitting: Example

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Lets assume Loves Popcorn is the best root node for the Decision tree that classifies Loves Cool As Ice

Loves Popcorn is True: Yes = 1 No = 3
Loves Popcorn is False: Yes = 2 No = 1

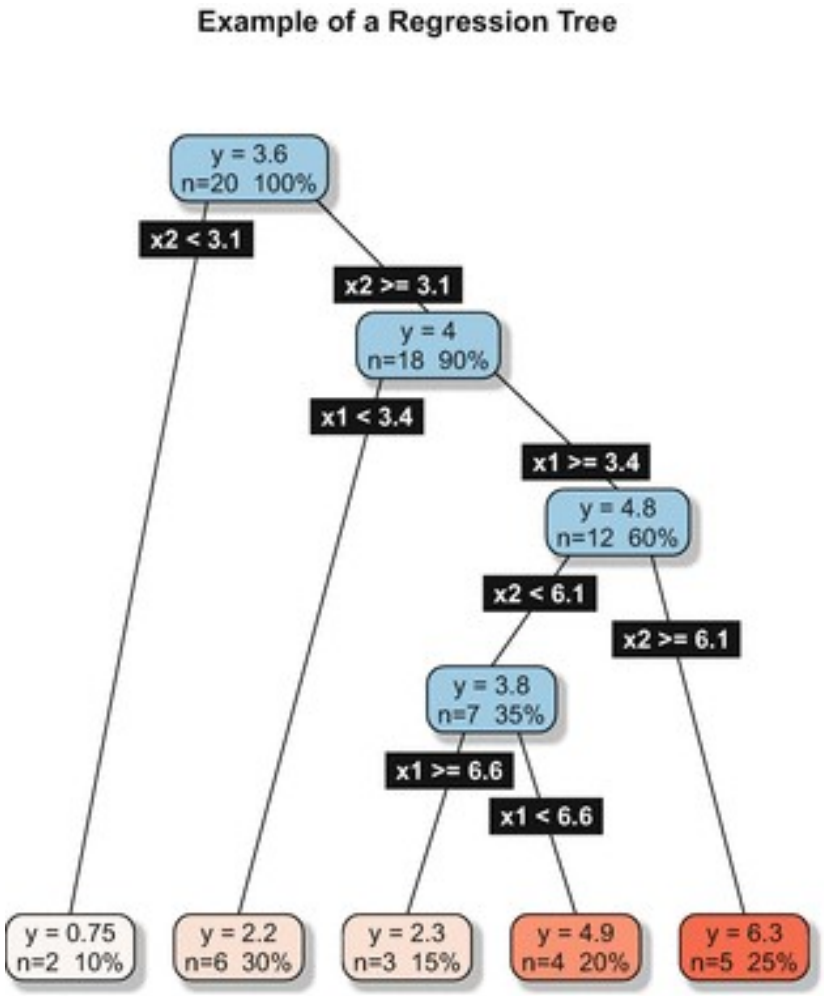
Gini Impurity for True:
1 – (the probability of Yes)^2 – (the probability of No)^2
= 1 – $(1/4)^2$ – $(3/4)^2$
= 0.375

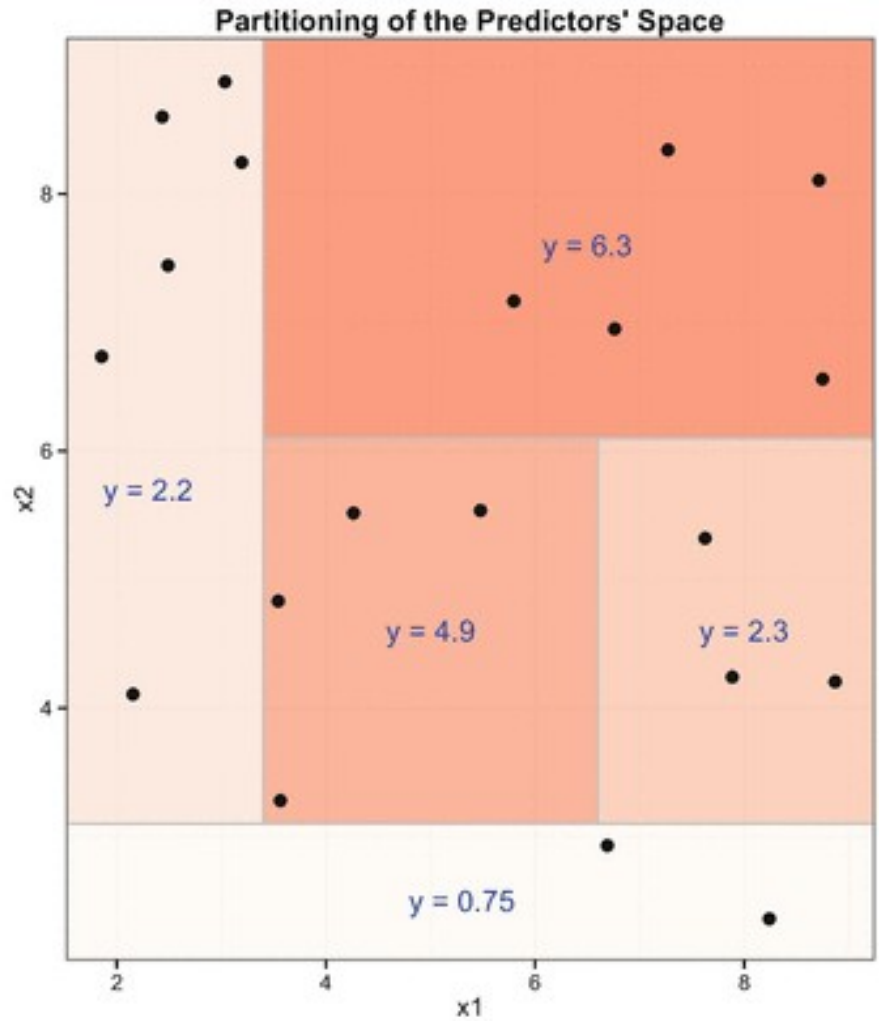
Gini Impurity for False: 0.444

Total Gini Impurity = weighted averages of Gini Impurities for the Leaves:
(4/7)*0.375 + (3/7)*0.444 = 0.405

THE LOWER THE BETTER:
The best split is the one that results in the **lowest weighted Gini impurity.**

Regression Tree





How to select the Best Feature for Splitting

In a **regression tree**, nodes (splits) are decided based on minimizing the variance (or another error metric) in the target variable. The goal is to create splits that result in the most homogeneous (least variable) child nodes.

Choose a Split Point
For each feature:
- Consider all possible split points (e.g., midpoints between consecutive values).
- Divide the dataset into two child nodes based on that split.

Calculate Split Quality (Variance Reduction)

$$\text{Variance Reduction} = \text{Variance(Parent Node)} - \left(\frac{N_{\text{left}}}{N}\text{Variance(Left Node)} + \frac{N_{\text{right}}}{N}\text{Variance(Right Node)}\right)$$

Where:

- N is the total number of samples in the parent node.
- N_{left}, N_{right} are the number of samples in each child node.
- Variance is computed as:

$$\text{Variance} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \bar{y})^2$$

where $\bar{y}$ is the mean target value in that node.

DT use **greedy algorithm** = makes the best choice at each step (locally optimal), without considering future consequences

How to select the Best Feature for Splitting

Select the Best Split
The split that results in the highest variance reduction (i.e., the most homogeneous child nodes) is chosen.

Recursion Until Stopping Criteria
Repeat the process recursively for child nodes until one of the stopping criteria is met:

- A **minimum number of samples** in a node (e.g., 5 samples).
- The **variance is below a threshold**.
- **Tree depth limit** is reached.

Tree Pruning

Unfortunately, the decision trees are prone to overfitting. To avoid it to introduce condition to stop growing is necessary.

Main reasons why to stop growing the tree:

- **All samples in a node belong to the same class** (pure node).
- **No more features left** to split.
- **Maximum tree depth** is reached (predefined).
- **Minimum samples per node** (predefined threshold) is reached.

Advantages

Easy to Understand & Interpret
Decision trees are intuitive and can be visualized easily, making them great for explaining results to stakeholders.

Handles Both Categorical & Numerical Data
Unlike some models that require numerical input, decision trees work with mixed data types.

No Need for Feature Scaling
Unlike models like logistic regression or SVM, decision trees don't require normalization or standardization.

Handles Non-Linearity Well
Can capture complex decision boundaries, unlike linear models.

Feature Selection Built-in
Automatically selects the most important features by splitting on the most informative variables first.

Fast Predictions
Once trained, a decision tree makes predictions quickly, making it efficient for real-time applications.

Handles Missing Values
Can work with missing data by assigning probabilities to different paths.

Disadvantages

Prone to Overfitting
If not pruned, trees can become too complex and memorize the training data rather than generalizing.

Sensitive to Noisy Data
Small changes in data can result in a completely different tree structure.

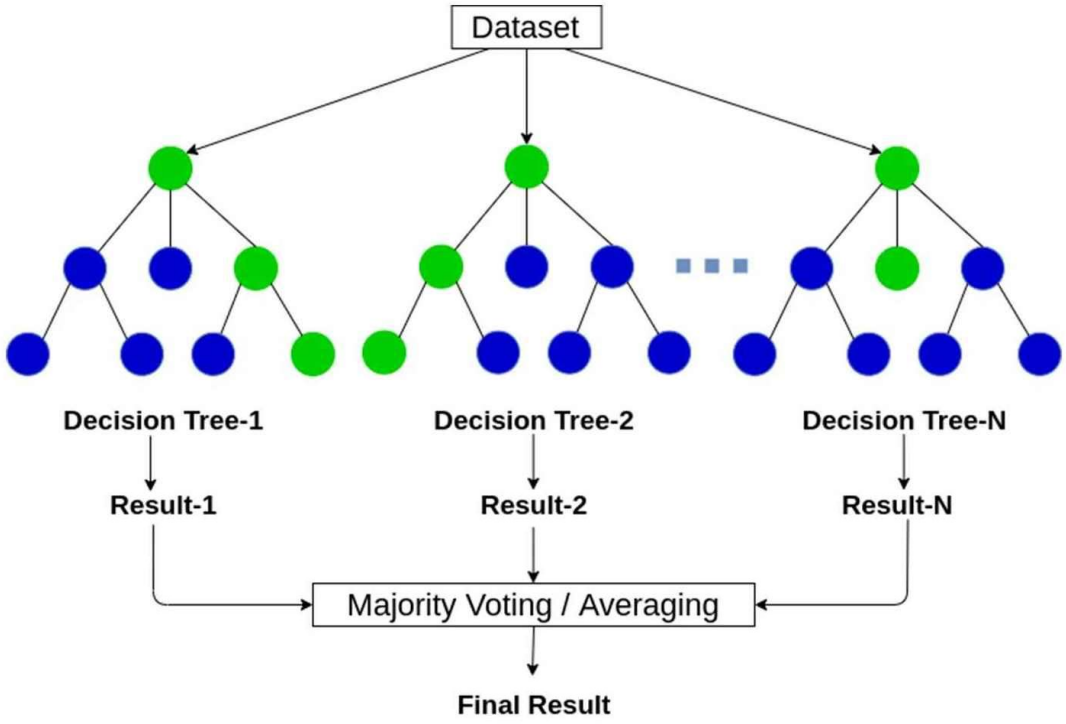
Biased Towards Dominant Classes
If one class dominates, the tree may favor it and make biased predictions.

Greedy Algorithm Might Not Find the Best Tree
Decision trees use a greedy approach (choosing the best split at each step), which doesn't always lead to the optimal solution.

Computationally Expensive for Deep Trees
Large trees require more memory and computation, which can slow down training.

Poor Performance on Small Data with Many Features
If the dataset is small but has many features, the tree may struggle to find meaningful splits.

The Anatomy of Random Forest



Random Forest is one of the most popular and powerful machine learning algorithms, used for both classification and regression tasks. It is an **ensemble learning method** that builds multiple decision trees and combines their predictions to improve accuracy and reduce overfitting.

Terminology Alert

Ensemble learning
Ensemble learning is a **machine learning technique** where multiple models (often called "weak learners") are combined to improve accuracy and robustness. Instead of relying on a single model, ensemble methods **aggregate predictions** to reduce errors and increase generalization. Bagging and Boosting are types of Ensemble learning

Bagging
Bagging is an ensemble learning technique that improves model stability and accuracy by **training multiple models on random subsets** of the dataset and averaging their predictions. It reduces **variance** and helps prevent **overfitting**.

Bootstrapping
Bootstrapping is a **resampling technique** used to estimate statistics (e.g., mean, variance) or improve model performance. It involves **randomly sampling data with replacement** to create multiple datasets.

Out-of-Bag Sample
The **Out-of-Bag (OOB) samples** refers to the data points **not selected** in a bootstrap sample when training models using **bagging** (Bootstrap Aggregating). These samples act like a **validation set** to evaluate model performance without using a separate test set. Since sampling is **with replacement**, some data points **are not selected** (~37% of the dataset).

Boosting
Boosting is a machine learning technique used to improve the accuracy of a model by combining the predictions of multiple weaker models to create a stronger one. The goal is to convert weak learners (models that perform slightly better than random guessing) into a strong learner that makes better predictions. Popular boosting methods: AdaBoost, Gradient Boosting. Advanced tools: XGBoost, LightGBM, CatBoost.

Building a Random Forest

1. Bootstrap Sampling (Bagging)
2. Random Feature Selection
3. Grow Decision Trees
4. Aggregation (Majority Voting/Averaging)
5. Model Evaluation / Hyperparameters tuning

Handling missing data in random forest (rfImpute in R)

- the missing data (NAs) are imputed with medians for numeric variables and modes for categorical variables.
- the random forest is is trained on completed dataset
- the proximity matrix from the model is then used to refine imputations
- ALERT: the response variable must not have any missing values

How it works:

Imagine the data for diabetes. You have measurements for blood sugar level, blood pressure, age, BMI, has diabetes (yes/no). You are missing some BSL data.

Step 1: Fill in missing values with rough estimates.
If a patient's glucose level is missing, we start by filling it in with the average glucose level of all other patients.

Step 2: Find Similar Patients.
We train a machine learning model (random forest) to learn patterns from the data. The model groups patients based on similar traits (e.g., BMI, age, and blood pressure). It creates a "map" showing how similar each patient is to others.

Step 3: Improve the Estimates.
Now, for the patient with missing glucose, we look at similar patients in our "map" and estimate their missing value based on those with complete information. If most similar patients have a glucose level around 120, we assume our patient's glucose is also close to 120. This process is repeated several times until the imputed values are as accurate as possible.

Bootstrap Sampling / Bagging

Original Dataset

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	No	Yes	167	Yes

Sampling with Replacement

Bootstrapped Dataset

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
Yes	Yes	Yes	180	Yes
No	No	No	125	No
Yes	No	Yes	167	Yes
Yes	No	Yes	167	Yes

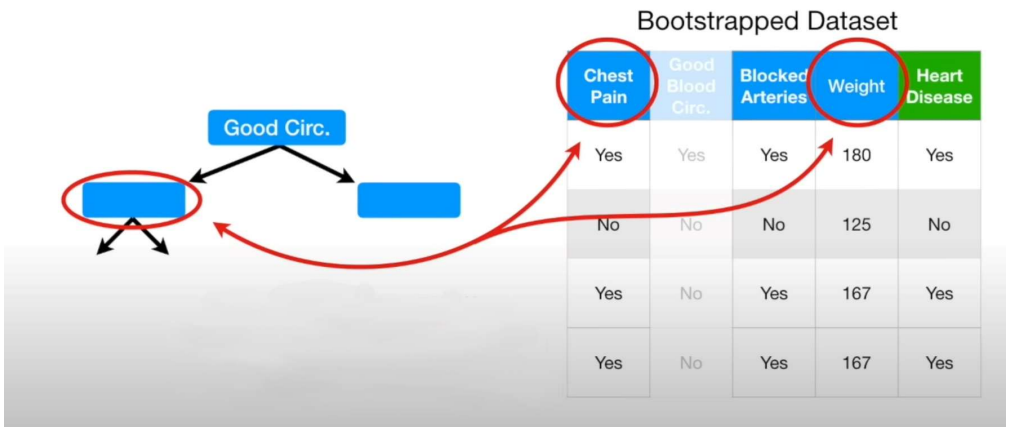
Multiple Samples
Generate multiple samples (called bootstrap samples) from the original dataset. Typically, the number of samples equals the size of the original dataset, but each sample may contain duplicates or lack some original data points.

Random Sampling
Each new sample is generated randomly, ensuring that each data point in the dataset has an equal probability of being included in any bootstrap sample.

Random Feature Selection

At each node of the decision tree, instead of considering all features (as in standard decision trees), the algorithm **randomly selects a subset of features**. This adds additional randomness and helps decorrelate the trees, making the final model more generalizable.

Each tree in the forest is grown to the maximum extent without pruning. The goal is to let each tree overfit its particular bootstrap sample.



Aggregation

For **classification**, the predictions of all trees are combined using **majority voting**. The class that gets the most votes becomes the final prediction.

For **regression**, the predictions of all trees are averaged to produce the final output.

Hyperparameter Tuning

Number of Trees (ntree)
The ntree parameter controls how many trees the random forest will include. More trees usually improve accuracy but also increase computational time.

Number of Features to Consider (mtry)
The mtry parameter determines how many features the algorithm should consider when making splits at each node.

Maximum Tree Depth (maxnodes)
R's randomForest function doesn't have a direct max_depth parameter, but you can control the depth of trees indirectly by adjusting the maxnodes parameter. This parameter limits the number of terminal nodes (leaf nodes).

Minimum Samples per Split (nodesize)
The nodesize parameter controls the minimum number of samples required to split an internal node. If there are fewer samples than the nodesize threshold at a node, it won't attempt to split the node further.

Minimum Samples per Leaf (nodesize)
nodesize also determines the minimum number of samples required to be at a leaf node. If there are fewer samples than the threshold at a leaf node, that node will not be further split, and it will be a terminal leaf.

Bootstrap Sampling (replace)
The replace parameter controls whether bootstrap sampling is used when building each tree. If replace = TRUE, then bootstrap sampling will be used (default behavior). If set to FALSE, then each tree is trained using the full dataset without sampling.

Hyperparameter Tuning

```
rf_model <- randomForest(Species ~ .,
                         data = train_data,
                         ntree = 500,          # Number of trees
                         mtry = 3,             # Number of features considered for each split
                         nodesize = 5,         # Minimum samples per node split
                         maxnodes = 30,        # Maximum number of terminal nodes
                         replace = TRUE,       # Use bootstrap sampling
                         importance = TRUE)    # Calculate feature importance
```

ntree = 500: Specifies 500 trees in the random forest.

mtry = 3: Specifies 3 features to consider at each split.

nodesize = 5: Specifies that a node must contain at least 5 samples to split.

maxnodes = 30: Limits the trees to have a maximum of 30 terminal nodes (leaf nodes).

replace = TRUE: Ensures bootstrap sampling is used when building each tree.

importance = TRUE: Calculates and displays feature importance, which can help identify the most important features for making predictions.

Summary

Aspect	Random Forest	Decision Tree
Nature	Ensemble of multiple decision trees	Single decision tree
Bias-Variance Trade-off	Lower variance, reduced overfitting	Higher variance, prone to overfitting
Predictive Accuracy	Generally higher due to ensemble	Less interpretable due to the ensemble
Robustness	More robust to outliers and noise	Sensitive to outliers and noise
Training Time	Slower due to multiple tree construction	Faster as it builds a single tree
Interpretability	Provides feature importance but less reliable	More interpretable as a single tree
Feature Importance	Provides feature importance scores	Provides feature importance but is less reliable
Usage	Suitable for complex tasks, high-dimensional data	Simple tasks, easy interpretation

Use DT When:

- You need interpretability (e.g., medical diagnosis).
- The dataset is small and not too complex.
- Fast predictions are required.

Use RF When:

- You need higher accuracy & robustness.
- The dataset is large and complex.
- You want to reduce overfitting.
- You need a more generalized model that performs well on new data.